

HashMap — 해시 테이블 기반 Key-Value 저장소

코딩테스트에서 빈도 세기, 존재 여부 확인, 그룹 분류에 핵심적으로 쓰이는 자료구조. 순서 없음, 키 중복 불허 (값은 중복 허용)

계층 위치: **Map** (interface) → **HashMap** (class) — Collection 계층과는 별도

핵심 메서드

메서드	설명	시간복잡도
<code>put(key, value)</code>	키-값 저장 (기존 키면 덮어씀)	O(1)
<code>putIfAbsent(key, value)</code>	키가 없을 때만 저장	O(1)
<code>get(key)</code>	키로 값 조회 (없으면 null)	O(1)
<code>getOrDefault(key, default)</code>	키로 값 조회 (없으면 기본값 반환, 원본 변경 없음)	O(1)
<code>merge(key, value, remapping)</code>	키 있으면 remapping 적용, 없으면 value 저장	O(1)
<code>containsKey(key)</code>	키 존재 여부	O(1)
<code>containsValue(value)</code>	값 존재 여부	O(n)
<code>keySet()</code>	모든 키의 Set 반환 (뷰)	O(1)
<code>values()</code>	모든 값의 Collection 반환 (뷰)	O(1)
<code>entrySet()</code>	모든 키-값 쌍의 Set 반환 (뷰)	O(1)
<code>size()</code>	저장된 쌍의 개수	O(1)
<code>isEmpty()</code>	비어있는지 확인	O(1)
<code>remove(key)</code>	키로 삭제	O(1)

ArrayList와 다르게 `set()` 메서드가 없는 이유

ArrayList는 `set(index, value)`로 값을 변경하지만, HashMap에는 `set()`이 없다. `put(key, value)`이 추가와 수정을 모두 처리하기 때문이다. 키가 없으면 새로 추가, 키가 이미 있으면 값을 덮어쓴다.

`getOrDefault` vs `putIfAbsent`

- `getOrDefault(key, default)`: 조회 결과에만 기본값을 반환. 원본 **HashMap**에는 아무것도 추가하거나 변경하지 않음
- `putIfAbsent(key, value)`: 키가 없을 때 실제로 맵에 저장까지 함. 키가 이미 있으면 아무것도 하지 않음

```
Map<String, Integer> map = new HashMap<>();

// getOrDefault - 맵에 "age" 키가 추가되지 않음
```

```
int val = map.getOrElse("age", 0); // 0 반환, map은 여전히 비어있음

// putIfAbsent - 맵에 "age" 키가 실제로 추가됨
map.putIfAbsent("age", 0); // map = {age=0}
```

merge 이해하기

merge = get + put을 하나로 합친 것

merge의 핵심은 **기존 값(old)**과 **새 값(new)** 두 개를 조합한다는 것이다. put은 값을 단순히 덮어쓰지만, merge는 기존 값을 참고하여 새 값과 조합하는 로직을 직접 정의할 수 있다.

```
// merge가 하는 일을 풀어쓰면 이렇다:
map.put(word, map.getOrElse(word, 0) + 1);

// merge로 한 줄:
map.merge(word, 1, Integer::sum);
// → 키가 없으면 1 저장, 있으면 기존값 + 1
```

:: (메서드 참조) 문법

`Integer::sum`에 사용된 `::`는 merge 전용 문법이 아니라 **Java 8에서 도입된 메서드 참조(method reference)** 문법이다. 람다를 짧게 쓸 수 있으며, 람다가 들어가는 곳이면 어디서든 사용 가능하다.

```
// 아래 두 줄은 완전히 같은 동작
map.merge(word, 1, (oldVal, newVal) -> oldVal + newVal); // 람다
map.merge(word, 1, Integer::sum);                        // 메서드 참조

// merge 외에도 람다가 들어가는 곳이면 어디서든 사용 가능
list.sort(Comparator.comparingInt(String::length));      // 정렬에서도 사용
list.forEach(System.out::println);                       // 출력에서도 사용
```

getOrElse + put vs merge 선택 기준

- 빈도수 세기처럼 **단순 누적**이면 `merge`가 간결
- 조건 분기가 복잡하면 `getOrElse + put`으로 풀어쓰는 게 읽기 편함

```
// 단순 누적 → merge가 간결
map.merge(word, 1, Integer::sum);

// 조건이 복잡한 경우 → getOrElse + put이 읽기 편함
int current = map.getOrElse(key, 0);
if (current > 10) {
    map.put(key, current + 5);
} else {
```

```
map.put(key, current + 1);
}
```

keySet()은 Set, values()는 Collection으로 반환하는 이유

- **keySet()** → Set: 키는 HashMap에서 **중복이 불가능**하므로, 중복을 허용하지 않는 Set으로 반환
- **values()** → Collection: 값은 **중복이 가능**하므로 (**{"a":1, "b":1, "c":2}** 처럼 값 1이 두 번 나올 수 있음), 중복을 허용하는 Collection으로 반환

시간복잡도 — 왜 대부분 O(1)인가

HashMap의 거의 모든 메서드가 O(1)인 이유는 해시 자료구조의 핵심 특징 때문이다. 키를 해시 함수에 넣으면 저장 위치(버킷)가 바로 계산되므로, 키 기반의 모든 연산이 O(1)이다.

- **containsValue()**만 O(n)인 이유: 값은 해시 함수로 위치를 역추적할 수 없어서 전체를 순회하며 찾아야 함 (키 → 값은 해시로 바로 가지만, 값 → 키 방향은 구조가 없음)
- **keySet(), values(), entrySet()**이 O(1)인 이유: 데이터를 복사하지 않고 원본 HashMap을 바라보는 **뷰(view)** 객체를 반환하기 때문 (subList와 같은 원리). 뷰를 수정하면 원본도 변경됨
- **size(), isEmpty()**: ArrayList와 마찬가지로 내부 필드로 개수를 관리

패턴

코딩테스트 출제 빈도: 패턴1 빈도 세기 > 패턴2 고유 개수 > 패턴3 부분 문자열 검사 > 패턴1심화 빈도 + 조합

패턴 1: 빈도 세기 — getOrDefault vs merge

언제 쓰는가

- 각 요소가 몇 번 등장하는지 카운트할 때
- 코딩테스트에서 가장 자주 나오는 HashMap 패턴
- **값(value)에 의미가 있는 패턴** — 값이 등장 횟수를 나타냄

실무에서는

- 로그 분석 (에러 코드별 빈도), 투표 집계, 장바구니 수량 관리

참고: Redis Sorted Set의 ZINCRBY

이 패턴은 Redis Sorted Set의 **ZINCRBY** 명령과 같은 맥락이다. 둘 다 "키가 이미 있으면 기존 값에 더하고, 없으면 새로 만든다"는 동작을 한다.

```
merge(key, 1, Integer::sum) ↔ ZINCRBY key 1 member
```

다만 Sorted Set은 score 기준 자동 정렬을 유지하는 구조이고, HashMap은 정렬 없이 순수하게 키-값만 저장한다는 차이가 있다.

```
import java.util.*;

public class FrequencyCounting {
    public static void main(String[] args) {
        String[] words = {"apple", "banana", "apple", "cherry", "banana",
"apple"};

        // === 방법 1: getOrDefault ===
        // 직관적이고 읽기 쉬움. 코딩테스트에서 가장 많이 쓰는 방식.
        Map<String, Integer> count1 = new HashMap<>();
        for (String word : words) {
            count1.put(word, count1.getOrDefault(word, 0) + 1);
        }
        System.out.println(count1); // {banana=2, cherry=1, apple=3}

        // === 방법 2: merge ===
        // 더 간결하지만 remapping 함수 이해가 필요.
        // merge(key, 1, Integer::sum) → 키가 없으면 1 저장, 있으면 기존값 + 1
        Map<String, Integer> count2 = new HashMap<>();
        for (String word : words) {
            count2.merge(word, 1, Integer::sum);
        }
        System.out.println(count2); // {banana=2, cherry=1, apple=3}
    }
}
```

방법 선택 기준

- 빈도수 세기처럼 단순 누적이면 **merge**가 간결
- 조건 분기가 복잡하면 **getOrDefault + put**으로 풀어쓰는 게 읽기 편함

```
// 단순 누적 → merge가 간결
map.merge(word, 1, Integer::sum);

// 조건이 복잡한 경우 → getOrDefault + put이 읽기 편함
int current = map.getOrDefault(key, 0);
if (current > 10) {
    map.put(key, current + 5);
} else {
    map.put(key, current + 1);
}
```

심화: 빈도 + 그룹별 조합 계산

언제 쓰는가

- 그룹별로 분류한 뒤 조합 수를 계산할 때
- "같은 종류끼리 짝짓기" 유형

- 패턴 1로 빈도를 센 뒤, 그 결과를 활용하는 패턴

실무에서는

- 상품 옵션 조합, A/B 테스트 그룹 배정

```
import java.util.*;

public class EntrySetCombination {
    public static void main(String[] args) {
        // 같은 색상의 옷 중 2개를 고르는 경우의 수
        String[] clothes = {"red", "red", "red", "blue", "blue", "green"};

        // 색상별 개수 세기 (패턴 1)
        Map<String, Integer> colorCount = new HashMap<>();
        for (String color : clothes) {
            colorCount.merge(color, 1, Integer::sum);
        }

        // 각 색상에서 2개를 고르는 조합 수:  $nC2 = n*(n-1)/2$ 
        int totalPairs = 0;
        for (Map.Entry<String, Integer> entry : colorCount.entrySet()) {
            int n = entry.getValue();
            if (n >= 2) {
                int pairs = n * (n - 1) / 2;
                System.out.println(entry.getKey() + ": " + n + "개 → " +
pairs + "쌍");
                totalPairs += pairs;
            }
        }
        // red: 3개 → 3쌍
        // blue: 2개 → 1쌍
        System.out.println("총 쌍의 수: " + totalPairs); // 총 쌍의 수: 4
    }
}
```

패턴 2: 중복 제거 + 고유 개수 — Set

언제 쓰는가

- 서로 다른 종류가 몇 개인지 셀 때
- "종류의 수"를 구하는 문제
- 키(key)에 의미가 있는 패턴** — 키의 존재 자체가 "이 종류가 있다"를 나타냄 (패턴 1은 값에 의미가 있었다면, 패턴 2는 키에 의미가 있음)

실무에서는

- 고유 사용자 수, 카테고리 목록 추출

```
import java.util.*;

public class UniqueCount {
    public static void main(String[] args) {
        String[] animals = {"cat", "dog", "cat", "bird", "dog", "fish"};

        // 기본: 고유 개수만 필요하면 HashSet
        Set<String> set = new HashSet<>(Arrays.asList(animals));
        System.out.println("종류 수: " + set.size()); // 종류 수: 4

        // 심화: 각 종류별 개수까지 필요하면 HashMap으로 빈도를 세고 keySet 크기
        Map<String, Integer> map = new HashMap<>();
        for (String animal : animals) {
            map.put(animal, map.getOrDefault(animal, 0) + 1);
        }
        System.out.println("종류 수: " + map.keySet().size()); // 종류 수: 4
        System.out.println("각 종류별 개수: " + map);           // {cat=2,
        dog=2, bird=1, fish=1}
    }
}
```

패턴 3: 부분 문자열 검사 — substring + 해시 O(1) 조회

언제 쓰는가

- 한 문자열이 다른 문자열의 부분 문자열(접두사, 접미사, 내부)인지 확인할 때
- 전화번호부 문제 등

이 패턴에서 HashMap/HashSet을 쓰는 이유

핵심은 키의 존재 여부를 **O(1)**로 빠르게 확인하기 위해서다. ArrayList의 `contains()`는 O(n)이지만, HashMap의 `containsKey()`나 HashSet의 `contains()`는 O(1)이다.

이 예제처럼 값(value)이 필요 없고 존재 여부만 확인하면 **HashSet이 더 적절**하다. 값에 부가 정보를 담아야 할 때 (예: 전화번호 → 사용자 이름) HashMap을 쓰면 된다.

```
import java.util.*;

public class SubstringCheck {
    public static void main(String[] args) {
        // 전화번호부에서 어떤 번호가 다른 번호의 접두사인지 확인
        String[] phoneBook = {"119", "97674223", "1195524421"};

        // 존재 여부만 확인하므로 HashSet이 더 적절
        Set<String> set = new HashSet<>(Arrays.asList(phoneBook));

        boolean hasPrefix = false;
        for (String phone : phoneBook) {
            // 한 글자씩 늘려가며 접두사가 존재하는지 확인
        }
    }
}
```

```

        for (int i = 1; i < phone.length(); i++) {
            String prefix = phone.substring(0, i);
            if (set.contains(prefix)) {
                System.out.println(prefix + "는 " + phone + "의 접두사입니
다");

                // 119는 1195524421의 접두사입니다
                hasPrefix = true;
            }
        }

        System.out.println("접두사 존재: " + hasPrefix); // 접두사 존재: true
    }
}

```

접두사 외에도 같은 방식으로 응용 가능

```

String phone = "1195524421";

// 접두사: 앞에서부터 자름
phone.substring(0, i);                // "1", "11", "119", ...

// 접미사: 뒤에서부터 자름
phone.substring(phone.length() - i);  // "1", "21", "421", ...

// 내부 문자열: 모든 구간을 자름
phone.substring(i, j);                // 가능한 모든 부분 문자열

```

주의사항

HashMap은 순서를 보장하지 않는다

```

Map<String, Integer> map = new HashMap<>();
map.put("C", 3);
map.put("A", 1);
map.put("B", 2);
System.out.println(map); // 순서가 삽입 순서와 다를 수 있음
// 순서가 필요하면 LinkedHashMap 사용
// LinkedHashMap = HashMap + 키들 사이에 삽입 순서대로 이중 연결 리스트(doubly linked list) 유지

```

Key의 equals/hashCode 관계

HashMap은 키를 찾을 때 두 단계를 거친다:

1. **hashCode()**로 버킷(저장 위치) 찾기: 키를 해시 함수에 넣어서 나온 정수값으로 어떤 버킷에 저장할지 결정

2. **equals()**로 정확한 키 찾기: 서로 다른 키가 같은 hashCode를 가질 수 있음 (해시 충돌). 같은 버킷에 여러 키가 들어갔을 때 정확히 어떤 키인지 **equals()**로 구분

```
// String, Integer 등은 hashCode()와 equals()가 이미 구현되어 있음
Map<String, Integer> map = new HashMap<>();
map.put("hello", 1);
map.get("hello"); // hashCode()로 버킷 찾고 → equals()로 "hello" 확인 → 1 반환

// 커스텀 객체를 키로 쓸 때는 둘 다 오버라이드 필수
// 하나라도 없으면 키 조치가 제대로 작동하지 않음
```

null 키와 null 값

```
Map<String, Integer> map = new HashMap<>();
map.put(null, 100); // null 키 1개 허용
map.put("key", null); // null 값 여러 개 허용
System.out.println(map.get(null)); // 100
System.out.println(map.get("key")); // null
```